Retro Save Sync

Design Doc v0.1 — FXPak Pro foundation, with extension points for EverDrives and emulator sync

Marke

April 25, 2026

Contents

1 Goals and non-goals **2**
1.1 Goals . . . 2
1.2 Non-goals (for v1) . . . 2

2 System overview **2**

3 Components **3**
3.1 FXPak Pro (current source) . . . 3
3.2 Raspberry Pi (sync hub) . . . 3
3.3 Google Drive (destination) . . . 3

4 Save change detection **4**
4.1 Polling loop . . . 4
4.2 Change classification . . . 4
4.3 Debounce window . . . 4
4.4 Why not file-system events / inotify? . . . 5

5 Versioning and backup strategy **5**
5.1 Path scheme in Drive . . . 5
5.2 Upload sequence . . . 5
5.3 Retention policy . . . 6
5.4 What “versioning” is *not* . . . 6

6 Restore flow **6**

7 State store schema **7**

8 Failure modes and how we handle them **7**

9 Security and privacy **8**

10 Implementation roadmap **8**

11 Appendix A — Future expansion **10**
11.1 Source-adapter contract . . . 10

11.2A.1 — EverDrives 10
11.3A.2 — Handheld emulator sync (RetroArch et al.) 10
11.4A.3 — Multiple Pis / multiple FXPak Pros 11

12 Appendix B — Open questions **11**

13 Appendix C — Component versions and references **12**

1 Goals and non-goals

1.1 Goals

- **Hands-off backup of SNES saves** from an FXPak Pro to Google Drive, without removing the SD card.
- **Versioned, recoverable history** — every distinct save state we observe is preserved, never overwritten in place. A specific past save can be restored by timestamp or hash.
- **A foundation that generalizes.** The same daemon, state store, naming scheme, and retention policy must work for other flash carts (EverDrives across N64 / GB / Mega Drive / etc.) and for handheld emulators (RetroArch and similar) when those are added later.
- **Operate on consumer hardware.** A Raspberry Pi 4 or 5 with a USB cable, a free Google account, and a residential internet connection are sufficient.
- **Fail safely.** Network outage, console power-off mid-write, or corrupt SRAM dumps must never destroy a known-good cloud version.

1.2 Non-goals (for v1)

- **Two-way sync.** v1 is upload-only from cart to cloud. Cloud-to-cart and emulator-to-cart sync are deferred to v2.
- **Real-time / push semantics.** A 30–90 second delay between in-game save and cloud upload is acceptable.
- **Multi-user / multi-tenant operation.** This is a single-operator system on a home network.
- **Web UI.** v1 ships with CLI tooling and structured logs only.

2 System overview

A Raspberry Pi sits between the FXPak Pro and the cloud. The cart stays plugged into the SNES; its USB port runs to the Pi. A long-lived daemon on the Pi periodically asks the cart for the list of save files, hashes each one, and uploads any that have changed to a versioned layout in Google Drive via `rclone`.

The diagram source is also provided as a standalone `save-sync-architecture.mermaid` file so it can be edited directly and re-rendered with `mmdc` (the Mermaid CLI) once the project repo is set up.

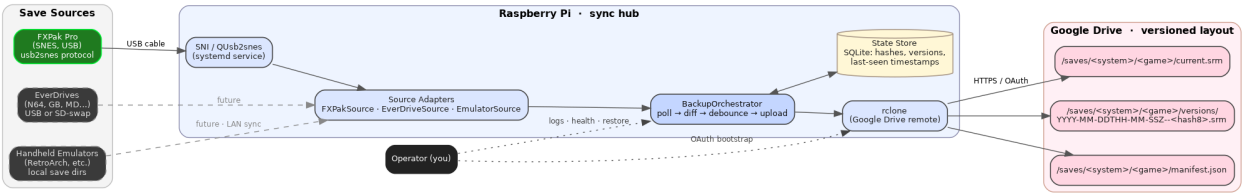

Figure 1: Architecture diagram

3 Components

3.1 FXPak Pro (current source)

The FXPak Pro stores SRAM as `.srm` files alongside the ROM on its FAT32 SD card. The cart's USB port speaks the **usb2snes** protocol — a small JSON-over-WebSocket protocol with commands like `List`, `GetFile`, `PutFile`, and `Info`. The protocol is well-documented and is what randomizer auto-trackers and multiworld tooling use today, so it is stable and battle-tested.

Cart-side semantics worth pinning down:

- The cart commits SRAM to SD on its own schedule — typically when the game itself writes SRAM, with a short flush window. We do not get a "save just changed" notification from the cart; we have to detect it ourselves.
- The cart can be queried while a game is running. There is no need to power-cycle the SNES.
- File listings include size and mtime, but **mtime should be treated as advisory only** — the FAT32 timestamps depend on cart firmware behavior. Hash is the source of truth.

3.2 Raspberry Pi (sync hub)

A Pi 4 or 5 running Raspberry Pi OS (64-bit). It hosts:

- **SNI** (Super Nintendo Interface) or **QUsb2snes**, run as a systemd service. SNI is the modern successor and is preferred. It exposes the usb2snes protocol on `ws://localhost:23074` (or `:8080` for QUsb2snes).
- **`save-syncd`** — the Python daemon described in this doc. Owns polling, hashing, debouncing, upload sequencing, and state.
- **`rclone`** with a Google Drive remote configured under a service-specific OAuth client.
- **SQLite state store** at `/var/lib/save-syncd/state.db`.
- **Structured logging** to `journald`, optionally shipped to a remote log sink later.

3.3 Google Drive (destination)

A single Drive folder, e.g. `My Drive/retro-saves/`, is the root. The daemon writes through `rclone` only; nothing else writes to this tree. Drive's own version history is left enabled as a secondary safety net, but our primary versioning lives in the path scheme so the layout is portable to any cloud provider rclone supports.

4 Save change detection

This is the most important behavioral spec in the doc — get it wrong and we either miss saves or thrash the upload queue.

4.1 Polling loop

Every **30 seconds** by default (configurable), `save-syncd`:

1. Connects to SNI at `ws://localhost:23074`.
2. Issues `Attach` for the FXPak Pro device, then `List /` recursively (or against a configured root).
3. Filters to files matching the configured "save file" predicate per source. For FXPak Pro, that is `*.srm` plus optionally `*.state` for cart-resident savestates.
4. For each candidate file:
   - Issues `GetFile` to read the bytes over USB. (Files are tiny — SNES SRAM is at most 32 KB; 64 KB for SA-1 carts. A full poll moves only a few hundred KB.)
   - Computes SHA-256 of the bytes.
   - Looks up the file's last known hash in SQLite.

4.2 Change classification

Given the new hash `H_new` and the previously stored hash `H_prev`:

Case	Condition	Action
Unchanged	`H_new == H_prev`	Update `last_seen_at`, do nothing else.
New file	No row for this `(source, path)`	Insert `pending_upload` row with `H_new`.
Changed	`H_new != H_prev`	Insert new `pending_upload` row with `H_new`; previous row stays as historical record.
Disappeared	File present last poll, absent now	Mark previous version `tombstoned` in DB; do **not** delete cloud copies.

4.3 Debounce window

A naive "upload on first detected change" would race the cart's flush behavior — we could read SRAM mid-write and capture a torn save. The daemon therefore applies a **stability debounce**:

> A `pending_upload` is only promoted to `ready_to_upload` once the file's hash has been observed unchanged for **N consecutive polls** (default `N = 3`, i.e. ~90 seconds). If the hash changes again during the debounce window, the timer resets and the new hash supersedes the previous pending one.

This trades latency (worst case ~2 minutes from in-game save to upload) for confidence that we never upload a torn write.

4.4 Why not file-system events / inotify?

We are not on the SD card directly — we are talking to the cart over usb2snes. There is no inotify-equivalent in the protocol. Future EverDrive sources that don't expose USB at all may rely on periodic SD-swaps, which is also poll-based. Polling is the lingua franca; we standardize on it.

5 Versioning and backup strategy

5.1 Path scheme in Drive

For each save we maintain three paths under a root folder (`retro-saves/`):

```
retro-saves/
└── <system>/
    └── <game-id>/
        ├── current.srm                     # latest known-good copy
        ├── manifest.json                   # rolling history index
        └── versions/
            ├── 2026-04-25T18-32-04Z--a3f1c290.srm
            ├── 2026-04-25T19-07-18Z--b88e0144.srm
            └── ...
```

Where:

- `<system>` is a stable identifier — `snes`, `n64`, `gb`, `gba`, `md`, `retroarch`. This is set by the source adapter, not derived from filename.
- `<game-id>` is a stable identifier per game. For FXPak Pro we use the ROM's CRC32 plus a slugified ROM filename: `0a1b2c3d_super_metroid`. The CRC is read from the cart's `Info` response or computed from the ROM at first encounter and cached. Slug-only would be ambiguous across regional revisions; CRC-only is unfriendly to humans.
- `versions/<timestamp>--<hash8>.srm` is the immutable record. Timestamp is UTC ISO-8601 with `:` swapped for `-` (Drive-safe). `<hash8>` is the first 8 hex chars of the SHA-256.
- `current.srm` is overwritten on every successful upload. It is a convenience pointer, not the source of truth — recovery always uses `versions/`.
- `manifest.json` is overwritten on every upload and contains the full version index plus pointers to retention status (see below). It exists so that listing is one read instead of N.

5.2 Upload sequence

Each upload is performed as a small atomic-ish ritual:

1. Write `versions/<ts>--<hash8>.srm` (`rclone copyto` from a local temp file).
2. Append the new entry to a locally-rebuilt `manifest.json`, then upload it.

3. Overwrite `current.srm`.
4. Commit the SQLite row state to `uploaded`.

If the daemon crashes between (1) and (4), recovery is straightforward: on startup, the daemon reconciles `manifest.json` against SQLite. Unindexed `versions/*` files are adopted into the manifest; SQLite rows in `uploading` state but absent from cloud are retried.

5.3 Retention policy

Drive storage is cheap but not free, and saves accumulate. The daemon runs a daily retention pass that classifies versions:

- **Last 24 hours:** keep every version.
- **Last 30 days:** keep one version per hour (the most recent in each UTC hour bucket).
- **Last 12 months:** keep one version per day.
- **Older than 12 months:** keep one version per week, indefinitely.

Retention only ever **moves entries** between buckets in the manifest, with one of two backend behaviors per version (configurable per game):

- **`prune`** — delete the cloud file. Default for high-frequency saves where intermediate states have no special meaning.
- **`archive`** — move to `versions/archive/` and exclude from default listings. Default for save files explicitly tagged "milestone" via a separate command.

Retention runs are recorded in SQLite so they are auditable and idempotent.

5.4 What "versioning" is *not*

We are deliberately not relying on Google Drive's built-in revision history as the primary mechanism, because:

- Drive caps revision history at 100 versions or 30 days for non-Workspace files (the limits and behavior have shifted historically — do not depend on them).
- It is invisible to `rclone` listings and tooling.
- It is not portable. If we ever re-target Backblaze B2, S3, or self-hosted storage, the path-scheme versioning works unchanged.

6 Restore flow

Restore is a manual operation in v1 — there is no scheduled "auto-pull" to the cart. The CLI exposes:

```
save-sync list   <system>/<game>              # show available versions
save-sync show   <system>/<game> <version>   # show metadata + size + hash
save-sync pull   <system>/<game> <version>   # download to local file
save-sync push  <system>/<game> <local>    # write to cart via usb2snes (PutFile)
```

Pushing to the cart is the only operation that mutates the cart and is gated by a `--confirm` flag plus a printed warning. The current cart-side `.srm` is captured to a `versions/` entry **before** the push, so a botched restore is itself recoverable.

7 State store schema

SQLite, single file, WAL mode. Tables:

```
CREATE TABLE sources (
  id          TEXT PRIMARY KEY,        -- 'fxpak-pro-1', 'retroarch-deck'
  system      TEXT NOT NULL,           -- 'snes', 'n64', ...
  adapter     TEXT NOT NULL,           -- 'fxpak', 'everdrive_n64', 'emulator_dir'
  config_json TEXT NOT NULL
);

CREATE TABLE files (
  source_id    TEXT NOT NULL REFERENCES sources(id),
  path         TEXT NOT NULL,           -- as reported by source
  game_id      TEXT,                    -- resolved game identifier
  first_seen   TEXT NOT NULL,
  last_seen    TEXT NOT NULL,
  current_hash TEXT,                    -- last hash we observed
  PRIMARY KEY (source_id, path)
);

CREATE TABLE versions (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  source_id     TEXT NOT NULL,
  path          TEXT NOT NULL,
  hash          TEXT NOT NULL,           -- full SHA-256
  size_bytes    INTEGER NOT NULL,
  observed_at   TEXT NOT NULL,           -- when daemon first saw this hash
  uploaded_at   TEXT,                    -- null until upload completes
  cloud_path    TEXT,                    -- 'snes/.../versions/2026-...'
  state         TEXT NOT NULL,           -- pending|debouncing|ready|uploading|uploa
  retention     TEXT NOT NULL DEFAULT 'keep',  -- keep|pruned|archived|milestone
  FOREIGN KEY (source_id, path) REFERENCES files(source_id, path)
);

CREATE INDEX versions_by_file ON versions(source_id, path, observed_at);
CREATE INDEX versions_pending ON versions(state) WHERE state IN ('pending','debounc
```

8 Failure modes and how we handle them

Failure	Detection	Behavior
Cart unplugged or SNES off	SNI Attach fails	Daemon backs off, retries every 30s. No state change.
Mid-write torn save	Hash differs from previous poll's torn read	Debounce window absorbs it; only stable hash promotes to upload.

Failure	Detection	Behavior
Upload fails (network)	rclone non-zero exit	Version stays in `ready_to_upload`; retried with exponential backoff up to 6h.
Drive auth expired	rclone 401 / token error	Daemon emits prominent warning to journald; stops upload attempts; polling continues so we don't lose change history.
Daemon crash mid-upload	Restart sees `uploading` rows	Reconcile against `manifest.json`; complete or roll back.
Disk full on Pi	SQLite write fails	Daemon halts with explicit error. No silent loss.
SRAM corruption on cart	We can't tell — bytes are bytes	Versioning is the safety net. Operator restores from a known-good `versions/` entry.

A daily health summary is written to journald and (optionally) emailed: last successful upload per game, total versions in the last 24h, retention actions, and any warnings.

9 Security and privacy

- **Google Drive scope:** the OAuth client requests `drive.file` only, which limits its access to files the app itself created. It cannot read the rest of your Drive.
- **Refresh token storage:** stored in rclone's config at `/home/save-sync/.config/rclone/rclon` owned by a dedicated `save-sync` system user, mode `0600`. The daemon runs as that user.
- **No secrets in the SQLite DB.** Hashes are not secrets.
- **Network exposure:** SNI binds to `127.0.0.1` only. The Pi does not expose any service to the LAN by default. SSH is the only inbound channel.
- **At-rest encryption (optional, off by default):** rclone supports a `crypt` backend wrapping the Drive remote. This adds operational complexity (lose the rclone passphrase → lose the saves) but is the right choice if backups will hold ROM-derived data the user wants encrypted at rest. Decision deferred to operator.

10 Implementation roadmap

Milestone 1 — bootable foundation (week 1) SNI installed and running on the Pi. Manual `rclone copy` of one `.srm` to Drive, end-to-end. Confirms the supply chain works.

Milestone 2 — daemon core (weeks 2–3) `save-syncd` polls FXPak Pro, hashes, debounces, uploads to the path scheme. SQLite state store. systemd service. No retention, no restore.

Milestone 3 — operator surface (week 4) `save-sync` CLI with `list / show / pull / push`, manifest reconciliation, daily health summary.

Milestone 4 — retention (week 5) Daily retention pass with the bucket policy above. `milestone` tagging.

Milestone 5 — hardening (week 6) Failure-mode coverage: simulated network outage, kill-9 mid-upload, torn-write injection. Backoff, recovery, alerting all proven.

After M5 the system is stable enough to layer the future expansion described in the appendix.

11 Appendix A — Future expansion

This section sketches how additional sources slot into the foundation without forcing a redesign. None of these are specified in detail yet; the purpose is to prove the v1 contracts won't have to change to accommodate them.

11.1 Source-adapter contract

Every source adapter implements:

```
class SaveSource(Protocol):
    id: str                                     # 'fxpak-pro-1'
    system: str                                 # 'snes', 'n64', ...

    async def health(self) -> HealthStatus: ...
    async def list_saves(self) -> list[SaveRef]: ...
    async def read_save(self, ref: SaveRef) -> bytes: ...
    async def write_save(self, ref: SaveRef, data: bytes) -> None: ...
    def resolve_game_id(self, ref: SaveRef) -> str: ...
```

The orchestrator only ever talks to this interface. The polling loop, debounce, hash diff, retention, and Drive layout are source-agnostic.

11.2 A.1 — EverDrives

EverDrive carts vary by system in how they expose saves:

- **EverDrive 64 X7 / GB Pro:** USB ports exist on some revisions, but firmware support for an FXPak-equivalent USB protocol is uneven. Where present, an `EverDriveSource` adapter wraps it.
- **Older or USB-less EverDrives (e.g. Mega Drive EverDrive X7):** v1 path is **SD-card swap with a card reader on the Pi** — adapter watches for a UDEV event when the card is inserted, mounts read-only, lists saves, hashes them, ejects. This becomes a manual-with-detection workflow rather than continuous polling.
- **Game identification** is per-system. N64 uses the internal ROM header CRC; GB uses the cartridge header title + hash; MD uses the ROM checksum field plus a hash of the first 4 KB. Each adapter brings its own `resolve_game_id`.

The path scheme already namespaces by `<system>` so cross-system collisions are impossible.

11.3 A.2 — Handheld emulator sync (RetroArch et al.)

Two sub-cases:

A.2.1 — Backup-only (trivial extension). A `LocalDirSource` adapter watches a directory (`~/.config/retroarch/saves/`) on the handheld, hashes files, uploads to the same Drive layout under `<system>/...`. Deck/Switch-emulating-handheld/etc. all become equivalent to "another source pointing at a directory". The orchestrator does not need to change.

A.2.2 — Two-way sync between handheld and FXPak Pro (the hard case). This is what makes the foundation matter. The flow:

1. You play *Super Metroid* on the FXPak Pro Saturday morning. The Pi syncs the new save to Drive.
2. You leave the house with the Steam Deck. RetroArch on the Deck pulls the latest save from Drive at launch.
3. You play on the Deck for two hours and save. RetroArch pushes the new save to Drive.
4. You come home and start *Super Metroid* on the SNES. The Pi pushes the Deck-derived save to the cart **before the SNES reads SRAM**.

The hard parts, none of which v1 has to solve:

- **Conflict detection.** Two devices may modify the save offline. We need a per-game "lineage" — each version records its parent hash. A version with no ancestor in the other device's recent history is a conflict.
- **Conflict resolution.** Default policy: preserve both as `versions/conflicts/<ts>--<hash8>.srm` and refuse to push to cart until the operator picks a winner via CLI. Last-writer-wins is the wrong default for save files because it silently destroys progress.
- **Cart-write timing.** Pushing to the FXPak Pro while the SNES is on but the game is not actively reading SRAM is fine for most games; pushing during gameplay can confuse SRAM-checksum logic. Adapter should only write when a "safe to write" precondition is met (SNES off, or game-specific allowlist).
- **Bootstrap on a fresh handheld.** A `save-sync init <system>/<game>` command pulls `current.srm` and primes the local emulator dir before launch. The handheld runs the same daemon; only the source type differs.

The v1 schema already supports this: the `versions` table records `hash` per version, and a future migration adds `parent_hash` and `lineage_id` columns. The Drive layout already has a `versions/` directory and a `manifest.json`; conflicts get a sibling `versions/conflicts/` directory and a `manifest.json` schema bump. No path-format breakage.

11.4 A.3 — Multiple Pis / multiple FXPak Pros

A second Pi (e.g. for a second console) registers a different `source.id` (`fxpak-pro-2`). Both write to the same Drive root with the same `<system>/<game>` namespace. The first uploader for a given hash wins; the second's `version` row is reconciled into the existing cloud entry. No coordination service is needed because content-addressing makes uploads idempotent.

12 Appendix B — Open questions

1. **SNI vs. QUsb2snes.** SNI is the modern path and supported by the Alttpr / multiworld communities, but QUsb2snes still has the larger install base in tooling. Pick one for v1; document the swap path. *(Recommended: SNI.)*
2. **Game ID for ROMs we've never seen.** First-time CRC computation requires reading the ROM over USB once, which is slower than an SRAM read. Worth caching

aggressively. Open question: does FXPak's `Info` command return the ROM CRC directly on current firmware? If yes, no read needed.

3. **Encryption of the Drive remote.** Default off, but should we recommend `rclone crypt` for users who care?
4. **Notification channel for the daily summary.** Email via `msmtp`, Discord webhook, or Pushover? Defaults vary by how technical the user is; pick a default and document the alternatives.
5. **Time source on the Pi.** Versioning depends on UTC timestamps. The Pi must run NTP (chrony preferred). Document as a hard prerequisite.

13 Appendix C — Component versions and references

- **SNI** — https://github.com/alttpo/sni
- **QUsb2snes** — https://github.com/Skarsnik/QUsb2snes
- **rclone Google Drive backend** — https://rclone.org/drive/
- **FXPak Pro firmware** — https://github.com/RedGuyyyy/sd2snes/releases
- **usb2snes protocol notes** — https://github.com/Skarsnik/QUsb2snes/blob/master/docs/Procotol.md